

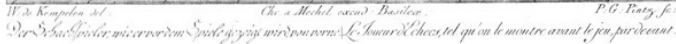
Autómatas

AUTÓMATA FINITO DETERMINISTA (AFD)

Contenido

Introducción

Un ejemplo de aplicación



Un ejemplo de autómatas: Dinero electrónico

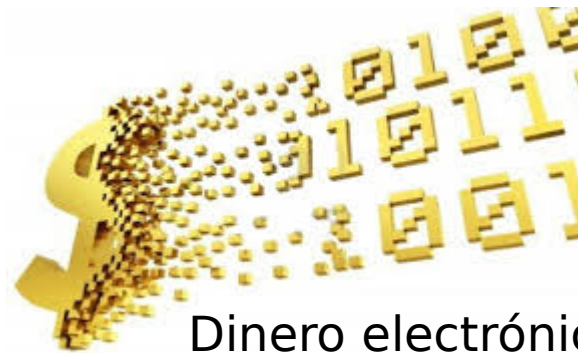
Tienda



Cliente



Banco



Dinero electrónico

Reglas

- Participantes: la cliente, la tienda, el banco.
- Solo hay un archivo de dinero electrónico.
- El cliente puede decidir transferir este archivo a la tienda la cual lo reenviará al banco (es decir, solicita al banco que emita un nuevo archivo que refleje que el dinero pertenece a la tienda y no más al cliente) y suministra los bienes al cliente.
- El cliente tiene la opción de cancelar el archivo (cancelar la compra); esto es, el cliente puede pedir al banco que devuelva el dinero a su cuenta, anulando la posibilidad de gastarlo.
- La interacción entre los participantes se limita a cinco sucesos:

Reglas

1. El cliente decide *pagar*. Es decir, el cliente envía el dinero a la tienda.
2. El cliente decide *cancelar* el pago. El dinero se envía al banco con un mensaje que indica que el dinero se ha añadido a la cuenta bancaria del cliente.
3. La tienda *suministra* los bienes al cliente.
4. La tienda *libra* el dinero. Es decir, el dinero se envía al banco con la solicitud de que su valor se asigne a la cuenta de la tienda.
5. El banco *transfiere* el dinero creando un nuevo archivo de dinero electrónico cifrado y se lo envía a la tienda.

El protocolo

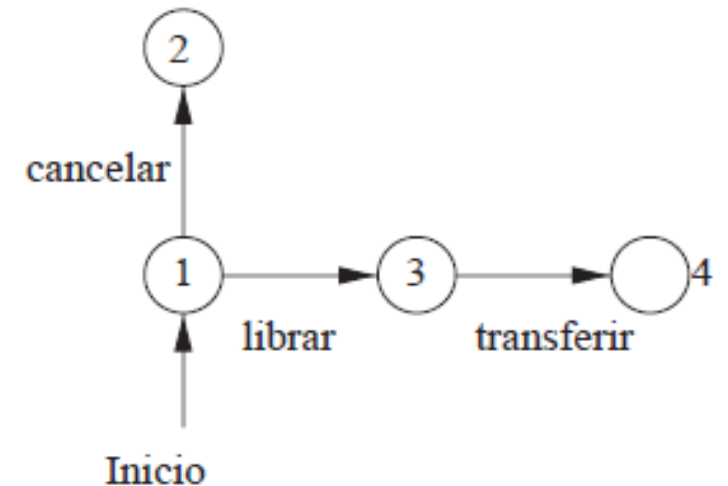
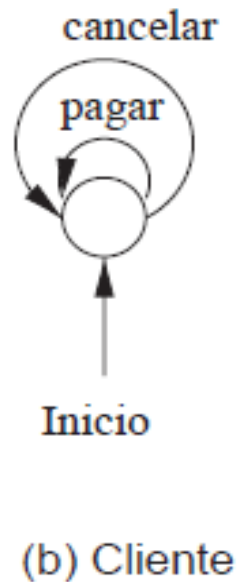
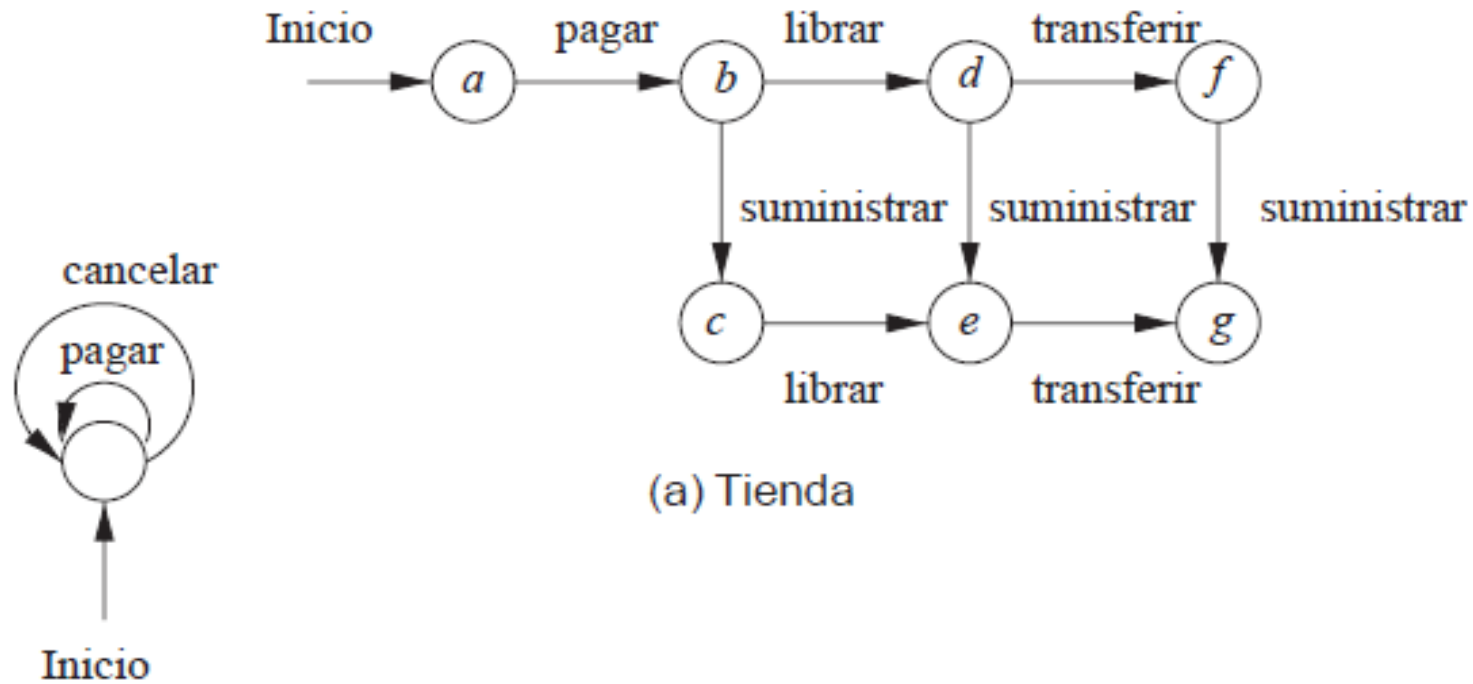
Los tres actores se deben comportar de forma responsable:

- La cliente no debe intentar copiar el archivo de dinero electrónico y emplearlo para pagar varias veces; tampoco debe pagar y cancelar el pago para recibir los bienes gratuitamente.
- El banco debe garantizar que dos tiendas no puedan liberar el mismo archivo de dinero; tampoco debe permitir que el mismo archivo de dinero sea a la vez cancelado por el cliente y liberado por la tienda.
- La tienda debe asegurarse de no suministrar bienes hasta haber recibido el dinero electrónico válido.

El protocolo

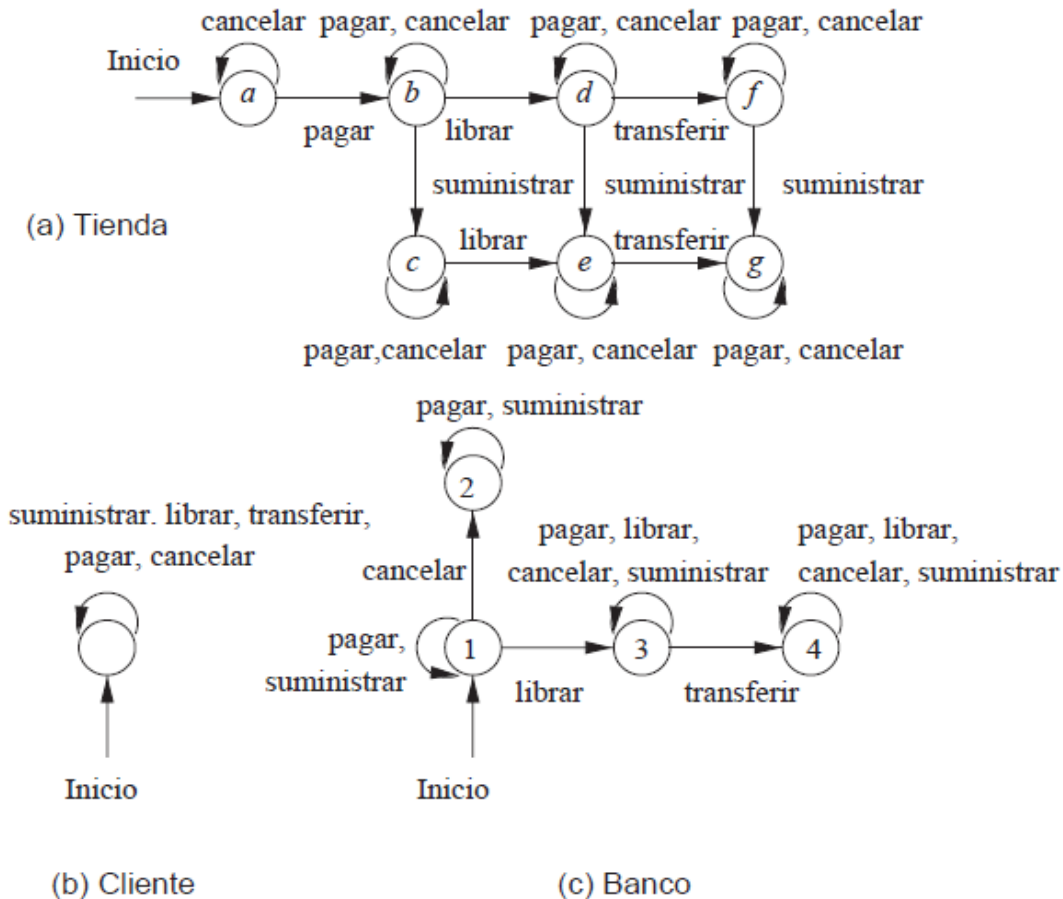
- Los protocolos pueden representarse de forma gráfica mediante un autómata finito
- Cada estado del autómata representa una situación en la que puede encontrarse uno de los participantes.
- Un estado “recuerda” que sucesos importantes han ocurrido y qué sucesos todavía no han tenido lugar.
- Las transiciones entre estados se producen cuando tiene lugar uno de los cinco sucesos descritos anteriormente.

Los autómatas para cada actor

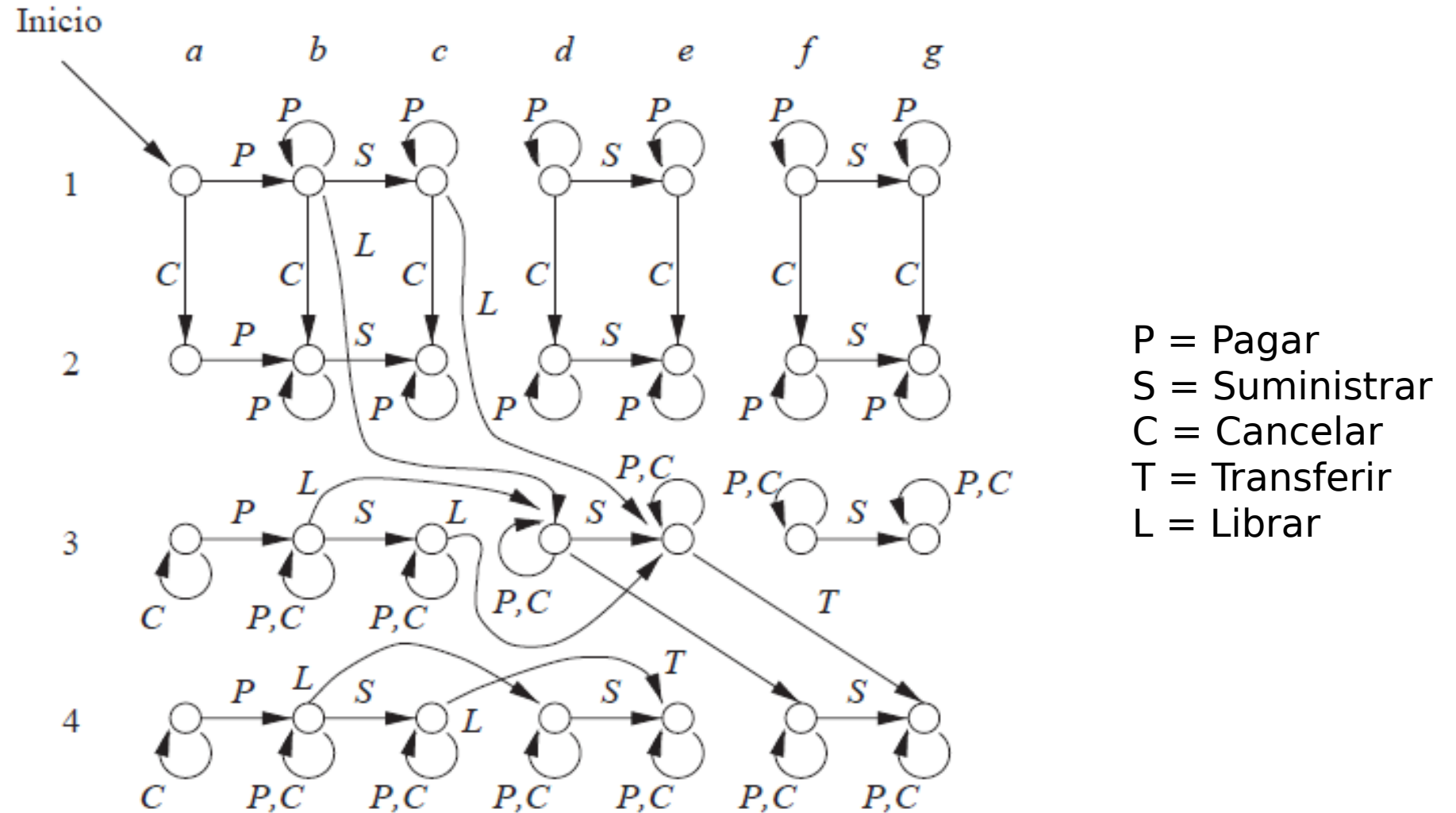


(c) Banco

Agregando transiciones adicionales (para evitar que el autómatata detenga su ejecución): acción *cancelar*.

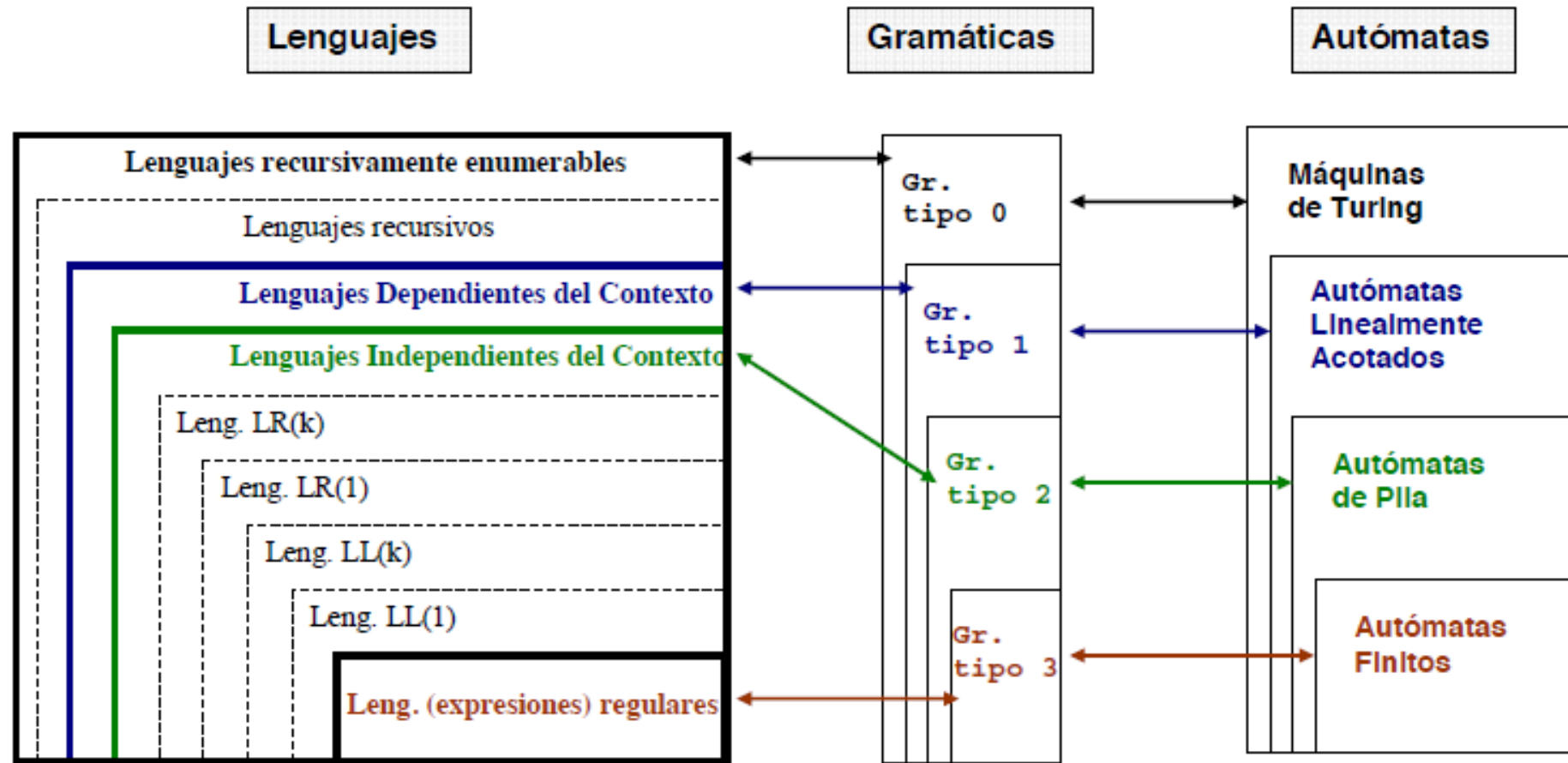


n autómata para el sistema completo



El autómata producto para la tienda y el banco

Contexto de nuestro curso de Teoría Computacional



Jerarquía de lenguajes, gramáticas y autómatas

Componentes de un compilador

- **Analizador léxico (scanner)**: Lee los caracteres del programa fuente, uno a uno, del archivo de entrada y va formando grupos de caracteres con alguna relación entre sí (*tokens*).

Tipos de tokens:

Cadenas específicas, como las palabras reservadas (*if, while, ...*), signos de puntuación (*., ,, =, ...*), operadores aritméticos (*+, *, ...*) y lógicos (*AND, OR, ...*), etc. Habitualmente, las cadenas específicas no tienen asociado ningún valor, sólo su tipo.

Cadenas no específicas, como los identificadores o las constantes numéricas o de texto. Las cadenas no específicas siempre tienen tipo y valor. Por ejemplo, si dato es el nombre de una variable, el tipo del token será *identificador* y su valor será *dato*.

- Los autómatas finitos son usados para el diseño de analizadores léxicos.
- El analizador léxico es una subrutina del analizador sintáctico.

Componentes de un compilador

- **Analizador sintáctico (*parser*)**: recibe como entrada los *tokens* generados por el analizador léxico y comprueba si estos tokens van llegando en el orden correcto.
- Si no hay errores, la salida del analizador sintáctico es un árbol semántico.
- Si el programa no es correcto se generarán los mensajes de error correspondientes.
- Los autómatas de pila son usados en el diseño de analizadores sintácticos.

Componentes de un compilador

- **Analizador semántico**: su propósito es determinar si el significado de las diferentes instrucciones del programa es válido.
- Para conseguirlo tendrá que calcular y analizar información asociada a las sentencias del programa:
 - Determinar el tipo de los resultados intermedios de las expresiones.
 - Comprobar que los argumentos de un operador pertenecen al conjunto de los operandos posibles.
 - Comprobar que los operandos son compatibles entre sí, etc.
- La salida de esta fase de compilación es un árbol semántico. Éste es una ampliación de un árbol sintáctico en el que cada rama del árbol ha adquirido, además, el significado que debe tener el fragmento de programa que representa.
- Esta fase del análisis es más difícil de formalizar que las dos anteriores y se utilizarán para ello las gramáticas atribuidas.

Componentes de un compilador

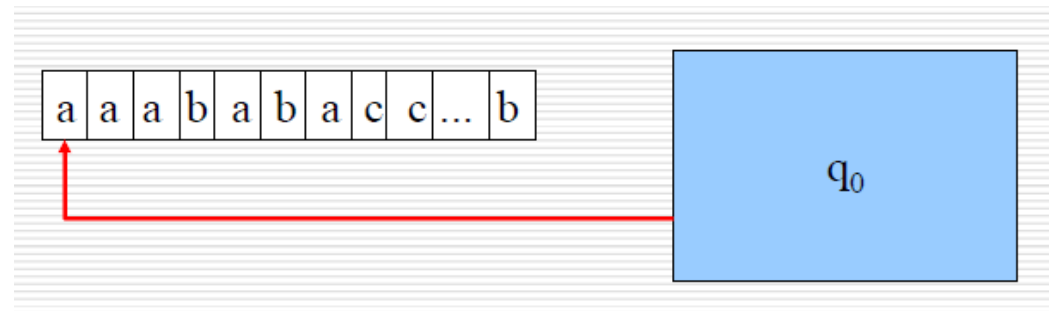
- Otras fases de compilación:
 - Generación de código intermedio.
 - Optimización de código.
 - Generación de código objeto.
 - Tabla de símbolos
 - Control de errores.

Autómatas finitos

- Un autómatas finito es un conjunto de estados y un control que se mueve de un estado a otro en respuesta a entradas externas.
- La salida de un autómatas finito está limitada a dos valores: *aceptado* y *no aceptado*, que pueden indicar si la cadena que se ha recibido como entrada es o no válida.
- Los autómatas finitos se pueden clasificar en función del tipo de control:
 - *Deterministas*, el autómatas únicamente puede estar en un estado en un momento determinado.
 - *No Deterministas*, el autómatas puede estar en varios estados simultáneamente.
- Utilizamos ambos para reconocer lenguajes regulares (indicar si una palabra pertenece a un lenguaje), sin embargo los **No deterministas** permiten describir más eficientemente determinados problemas.

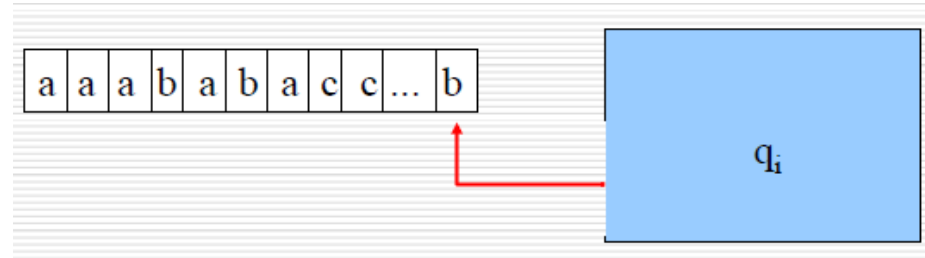
Autómata finito determinista (AFD)

- ¿Cómo procesa las entradas un AFD?
 - La entrada a un AF es un conjunto de símbolos tomados del alfabeto de entrada Σ , no hay límite en tamaño de la cadena.
 - Existe un “puntero” que en cada momento apunta a una posición de la cadena de entrada.
 - El autómata está siempre en un estado de Q , inicialmente se encuentra en el estado q_0 .



Autómata finito determinista (AFD)

- ¿Cómo procesa las entradas un AFD?
 - En cada paso el autómata lee un símbolo de la entrada y según el estado en el que se encuentre, cambia de estado y pasa a leer otro símbolo.



- Y así sucesivamente hasta que se terminen de leer todos los símbolos de la cadena de entrada.
- Si en ese momento el AF está en un estado q_i de F (conjunto de estados finales), se dice que acepta la cadena, en caso contrario la rechaza.

AFD – Definición formal

Un AFD se define como una quintupla:

$$AFD = (\Sigma, Q, f, q_0, F)$$

donde:

Σ es el alfabeto de entrada.

Q es el conjunto finito y no vacío de los estados del Autómata.

f es la función de transición que indica en qué situaciones el Autómata pasa de un estado a otro. se define:

$$f : Q \times \Sigma \longrightarrow Q$$

$q_0 \in Q$ es el estado inicial

$F \subset Q$ es el conjunto de estados finales de aceptación ($F \neq \emptyset$)

Representación de AFD

Tablas de transición

- Es una representación clásica de una función con dos argumentos.
- En las filas se colocarán los estados y en las columnas los símbolos del alfabeto de entrada.
- Cada intersección **fila (estado q) - columna (carácter a)** corresponde al estado **$f(q,a)$** .
- El estado inicial se representa con $\rightarrow q_0$
- Los estados finales se representan con $*q_i$

Ejemplo:

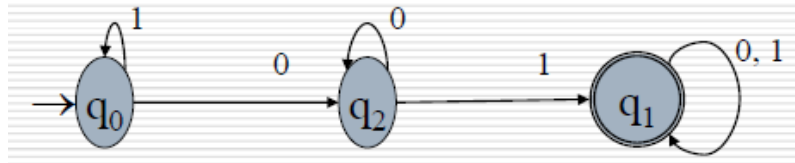
	0	1
$\rightarrow q_0$	q_2	q_0
$*q_1$	q_1	q_1
q_2	q_2	q_1

Representación de AFD

Diagramas de transición

- Es un grafo en el que los vértices representan los distintos estados y los arcos las transiciones entre los estados.
- Cada arco va etiquetado con el símbolo que corresponde a dicha transición.
- El estado inicial se representa con 
- Los estados finales con un doble círculo.

Ejemplo:

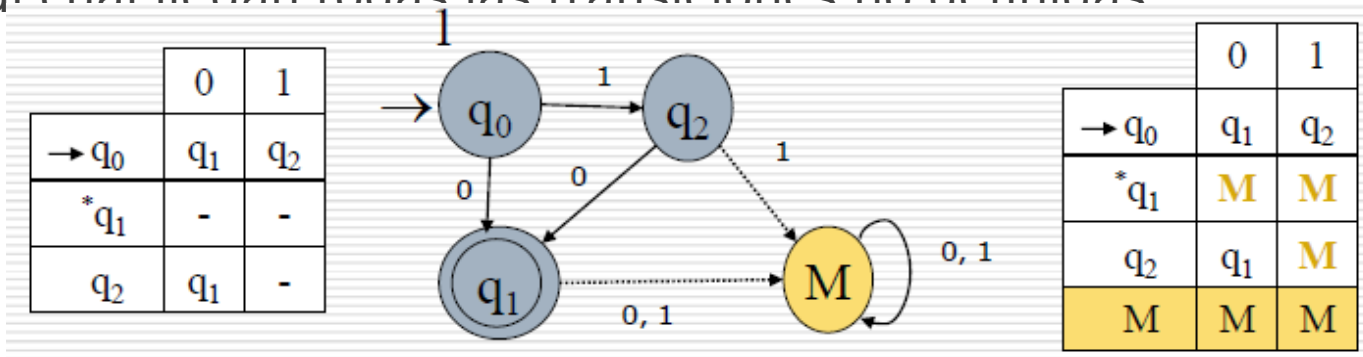


Representación de AFD

□ Determinismo porque:

- No existen transiciones λ
- $\forall q \in Q, \forall a \in \Sigma$ una única $f(q, a)$:
 - Una sola arista etiquetada con a para cada símbolo;
 - Para cada entrada en la tabla un solo estado
- La indeterminación en el caso que falten transiciones para algunas entradas se resuelve incluyendo un nuevo estado, llamado de **absorción** o **muerto**, al cual llegan todas las transiciones no definidas

▪ Ejemplo:



Ejemplo AFD

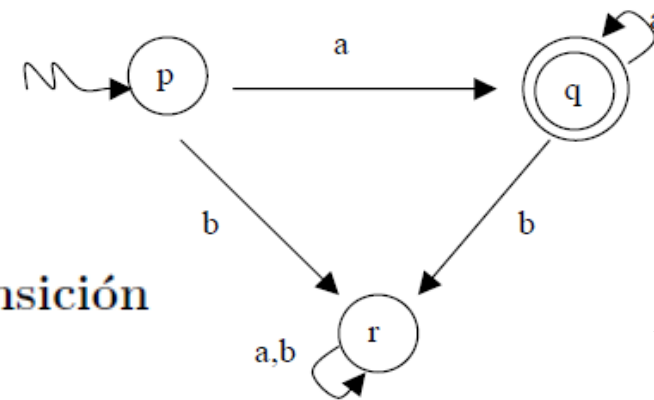
Sea el siguiente AFD: $\Sigma = \{a, b\}$ $Q = \{p, q, r\}$ $q_0 = p$ $F = \{q\}$
donde f se define de la siguiente forma:

$$\begin{array}{ll} f(p, a) = q & f(p, b) = r \\ f(q, a) = q & f(q, b) = r \\ f(r, a) = r & f(r, b) = r \end{array}$$

Tabla de transición El AFD se representaría mediante la siguiente tabla que representa los valores de la función de transición.

		<i>a</i>	<i>b</i>
→	<i>p</i>	<i>q</i>	<i>r</i>
*	<i>q</i>	<i>q</i>	<i>r</i>
	<i>r</i>	<i>r</i>	<i>r</i>

Diagrama de transición



¿Qué cadenas del Alfabeto Σ pueden ser identificadas?

Ejemplo AFD

Dado el lenguaje $L = \{ x \mid x \text{ en } \{0, 1\}, x \text{ comienza con } 0 \}$, es decir todas las cadenas binarias que comienzan con 0. El AFD que lo reconoce es el siguiente:

$M = (Q, \Sigma, f, q_0, F)$, $\Sigma = \{0, 1\}$, $F = \{q_1\}$, $Q = \{q_0, q_1, q_2\}$

Función de transición f Tabla de transiciones

$$f(q_0, 0) = q_1$$

$$f(q_0, 1) = q_2$$

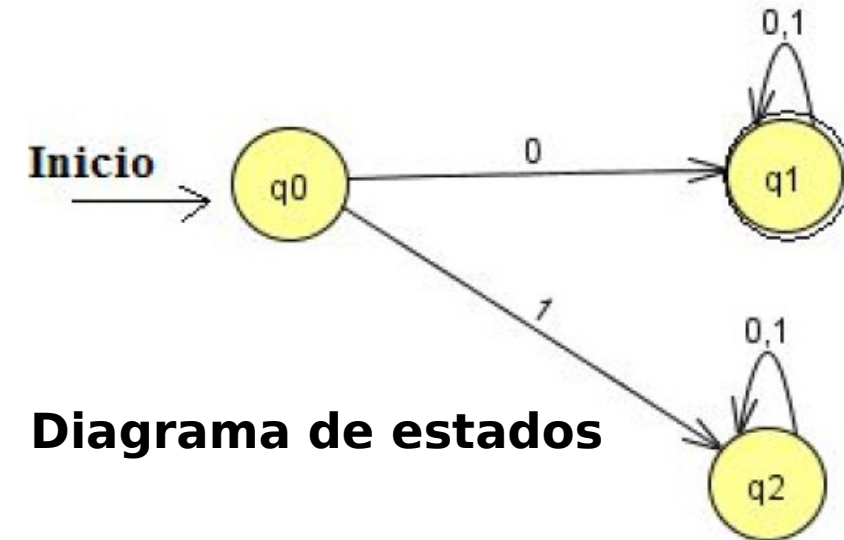
$$f(q_1, 0) = q_1$$

$$f(q_1, 1) = q_1$$

$$f(q_2, 0) = q_2$$

$$f(q_2, 1) = q_2$$

f	0	1
--> q ₀	q ₁	q ₂
*q ₁	q ₁	q ₁
q ₂	q ₂	q ₂



Ejecución de un AFD

Para la ejecución formal de un AFD se utiliza la definición de la función de transición extendida

sobre cadenas: ***f***

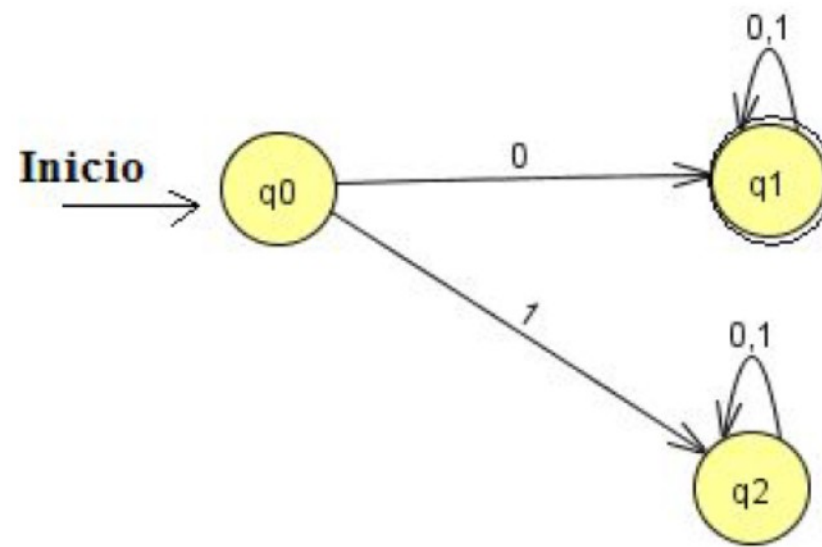
$$f: Q \times \Sigma^* \rightarrow Q$$

$$f(q, \lambda) = q \quad \forall q \in Q$$

$$f(q, ax) = f(f(q, a), x) \quad a \in \Sigma, x \in \Sigma^*, q \in Q$$

f se usa de forma recursiva

f	0	1
$\rightarrow q_0$	q_1	q_2
$*q_1$	q_1	q_1
q_2	q_2	q_2



Por ejemplo: Dado el AFD anterior que reconoce el lenguaje $L = \{ x \mid x \text{ en } \{0,1\}^*, x \text{ comienza con } 0 \}$, pruebe si las cadenas $w1 = 010$ y $w2 = 101$ están en el lenguaje L .

□ **$f(q_0, 010)$** $= f(f(q_0, 01), 0) = f(f(f(q_0, 0), 1), 0) = f(f(f(f(q_0, \lambda), 0), 1), 0) \rightarrow f(f(f(q_0, 0), 1), 0) \rightarrow$

$f(f(q_1, 1), 0) \rightarrow f(q_1, 0) = q_1 \rightarrow$ **“Acepto”** al terminar de procesar la cadena y quedar en un

estado de aceptación (esta en F , es decir, en q_1).

□ **$f(q_0, 101)$** $= f(f(q_0, 10), 1) = f(f(f(q_0, 1), 0), 1) = f(f(f(f(q_0, \lambda), 1), 0), 1) \rightarrow f(f(f(q_0, 1), 0), 1) \rightarrow$

Lenguaje aceptado por un AFD

Sea $M = (Q, \Sigma, f, q_0, F)$ un AFD, el lenguaje **L** aceptado por **M** esta definido por:

$$L(M) = \{ w \mid w \in \Sigma^* \text{ y } f(q_0, w) \in F \}$$

Nota: el lenguaje de **M**, es un conjunto de cadenas **w** que llevan al autómata **M** desde q_0 a un

estado de aceptación. Si **L** es un **L(M)** para algún AFD **M**, entonces decimos que **L** es un

“Lenguaje Regular”.

Nota importante: Para poder verificar si la cadena **w** está (es aceptada) o no está(no es

aceptada) en **L** se debe procesar por completo.

Ejemplo

Dado el lenguaje $L = \{ w \mid w \text{ tiene un numero par de ceros y unos} \}$

$L = \{ 00, 11, 1100, 0011, 0101, 1010, 001010, \dots \}$

$M = (Q, \Sigma, f, q_0, F)$

$\Sigma = \{0,1\}$, $Q = \{q_0, q_1, q_2, q_3\}$

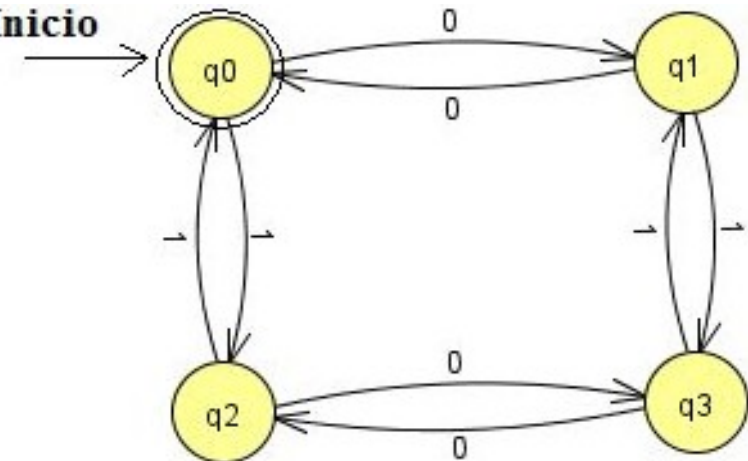
$F = \{ q_0 \}$

Para la cadena 0101 probar el AFD:

$f(q_0, 0101) = f(f(q_0, 010), 1) = f(f(f(q_0, 01), 0), 1) = f(f(f(f(q_0, 0), 1), 0), 1) =$

$f(f(f(f(f(q_0, \lambda), 0), 1), 0), 1) \rightarrow f(f(f(f(q_0, 0), 1), 0), 1) \rightarrow f(f(f(f(q_1, 1), 0), 1) \rightarrow f(f(q_3, 0), 1) \rightarrow f(q_2, 1)$

$= q_0$, $\rightarrow$ **"Acepto"** , al terminar de procesar la cadena y quedar en el estado de aceptación q_0 (q_0 esta en F , es verdadero)



Ejercicios con AFD

Para $\Sigma = \{0,1\}$ construya un AFD que acepte los siguientes lenguajes:

- a) El conjunto de cadenas que tienen el mismo número de unos y de ceros:
0101, 000111, ...
- b) El conjunto de cadenas en las que el numero de ceros es divisible por 3.
- d) El conjunto de cadenas que terminan con 01.
- f) El conjunto de cadenas que comienzan con 10
- e) El conjunto de cadenas que tienen un número impar de unos y de ceros.

Simulación algorítmica de un AFD

- ❑ Entrada: cadena de entrada x que termina con un carácter fin de cadena o fin de archivo (FDC).
- ❑ Salida: La respuesta ACEPTADA si el autómata reconoce; NO ACEPTADA en caso contrario
- ❑ Método: aplicar f al estado al cual hay una transición desde el estado q a un carácter de entrada c

Función reconocer()

$q = q_0$

$c = \text{leer_carácter}()$

mientras $c \neq \text{FDC}$

$q = f(q, c)$

$c = \text{leer_carácter}()$

fmientras

Si $q \in F$ entonces

Devolver(ACEPTADA)

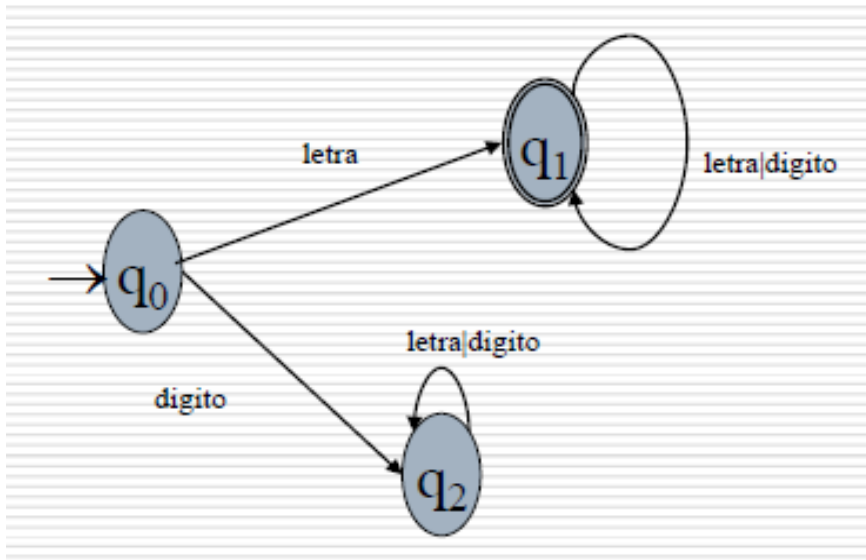
Sino

Devolver(NO ACEPTADA)

fsi

Simulación algorítmica de un AFD

Ejemplo: AFD que reconozca el nombre de variables empezando por letra y seguida de letras o dígitos.



	letra	dígito
→ q ₀	q ₁	q ₂
* q ₁	q ₁	q ₁
q ₂	q ₂	q ₂

Simulación algorítmica de un AFD

```
Función reconocer_id() //Por el diagrama
estado=0
s= leer_simbolo()
Mientras s != FDC
    Caso estado sea
        0: Si s=letra entonces estado=1
            sino
                Si s=digito entonces estado=2
                    sino
                        Error() //salir a la rutina de error léxico
                    fsi
                fsi
            fsi
        1: Si s=letra or digito entonces estado=1
            sino
                Error() //salir a la rutina de error léxico
            fsi
        2: Si s=letra or digito entonces estado=2
            sino
                Error() //salir a la rutina de error léxico
            fsi
    fcaso
s= leer_simbolo()
fmientras
Si estado=1 entonces
    Devolver(SI)
sino
    Devolver(NO)
fsi
```

Simulación algorítmica de un AFD

Función reconocer_id() //por la tabla

estado=0

s= leer_simbolo()

Mientras s != FDC

Caso s sea

letra: entrada=letra

digito: entrada=digito

otro: Error() //salir a la rutina de error léxico

fcaso

estado= f(estado,entrada)

s= leer_simbolo()

fmientras

Si estado=1 entonces

Devolver(SI)

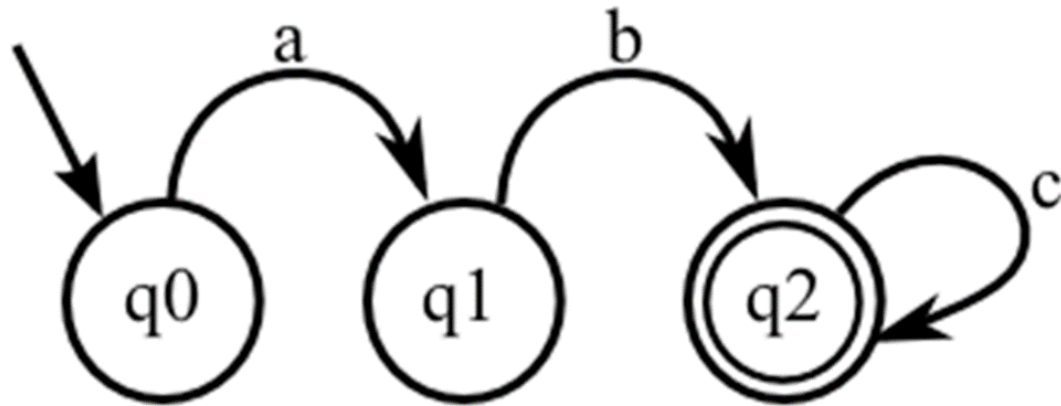
sino

Devolver(NO)

fsi

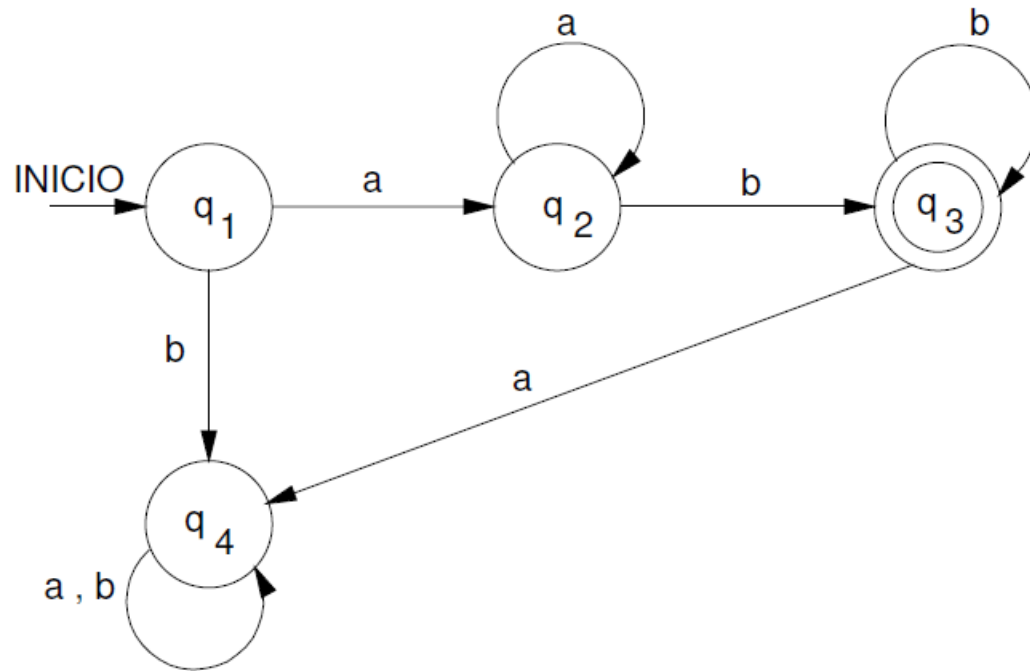
AFD y Expresiones regulares

Un autómata finito es una maquina de estados capaz de reconocer un **lenguaje regular**, si el autómata es capaz de alcanzar en su configuración final un estado del conjunto F.



abc*

Ejemplo 1



$$L(A) = \{ab, aab, \dots, abbb, \dots, aabb, \dots\}$$

$$L(A) = \{ a^n b^m \mid n \geq 1, m \geq 1 \}$$

La expresión regular que denota el lenguaje es a^+b^+ o también aa^*bb^* .

Ejemplo 2

Los identificadores de C son: cadenas de letras, dígitos y guiones bajos.

letra_ $\rightarrow$ [A-Za-z_]

dígito $\rightarrow$ [0-9]

id $\rightarrow$ *letra_* (*letra_* | *dígito*)*

El autómata finito es el siguiente:

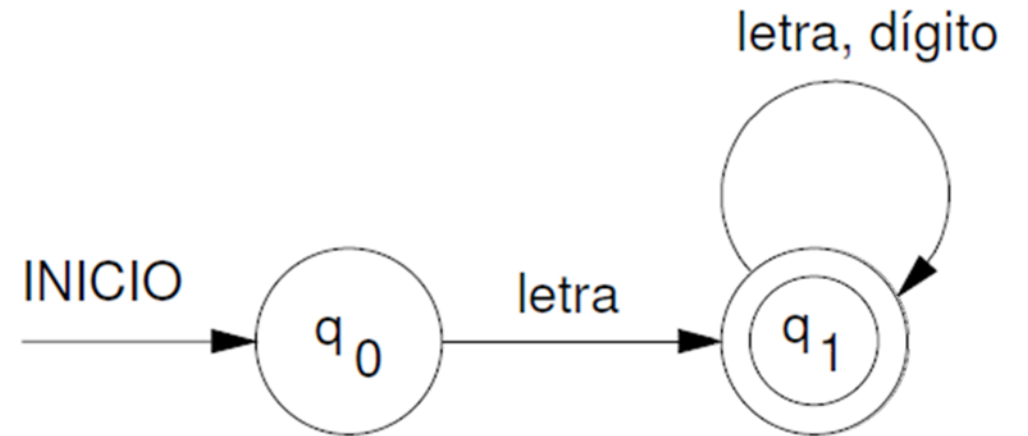
$AFD = \{ \Sigma = \{a, b, \dots, z, 0, 1, \dots, 9, \$\}, Q = \{q_0, q_1, q_2\}, f, q_0, F = \{q_2\} \}$

\$ = fin de cadena

Ejemplo 2 - continuación

Función f :

f	<letra>	<dígito>	\$
q_0	q_1	-	-
q_1	q_1	q_1	q_2
q_2	-	-	-



Diseña autómatas finitos capaces de reconocer los siguientes lenguajes definidos por la expresiones regulares:

1. $a+b^*$
2. $ab(cd)^+e$
3. $(a+b)^+cd$
4. $abz?b^+$
5. $(cg)^*gato^+ | cd$